

Proyecto I: Explorando los Algoritmos de Aprendizaje Supervisado

1st Luis Felipe Jimenez Ugalde

Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

luijimenez@estudiantec.cr

2nd Carlos Eduardo Rodríguez Segura

Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

c.rodriguez@estudiantec.cr

Abstract—This work evaluates different machine learning algorithms through three case studies: logistic regression and decision trees for income prediction, linear regression and random forest for regression tasks, and k-nearest neighbors with random forest for image classification. Both manual implementations and optimized scikit-learn models were compared. Preprocessing included handling missing values, resampling to balance data, and dimensionality reduction with PCA. Results show that logistic regression generalized better than decision trees, random forest outperformed linear regression, and random forest with PCA achieved the best classification accuracy. While custom implementations reproduced expected behavior, scikit-learn provided superior efficiency and slightly better results.

Index Terms—Machine Learning, Classification, Regression, Logistic Regression, Decision Trees, Random Forest, K-Nearest Neighbors (KNN), Model Optimization

I. INTRODUCCIÓN

En el presente trabajo se desarrollan e implementan diferentes modelos de aprendizaje supervisado tanto en su versión implementada desde cero, como utilizando la librería sklearn. El objetivo principal es comprender el funcionamiento interno de los algoritmos, contrastar su desempeño a través de diferentes métricas con implementaciones profesionales y analizar las ventajas y limitaciones de cada enfoque.

El proyecto se divide en tres problemas con set de datos diferentes cada uno:

- **Regresión logística y árboles de decisión:** Se desarrolla una regresión logística basada en gradiente descendente y un árbol de decisión para clasificación. Además, se utilizan técnicas de optimización de hiperparámetros mediante GridSearchCV para encontrar la mejor configuración tanto en los modelos propios como en los de sklearn.
- **Regresión lineal y Random Forest:** Se incluye la implementación manual de la regresión lineal utilizando mínimos cuadrados ordinarios, así como un árbol de decisión adaptado a problemas de regresión. Con base en ello se construye un Forest propio y se comparan sus resultados frente a las implementaciones estándar de sklearn. También se evalúan diferentes métricas de error (MAE, MSE, RMSE y R^2).
- **Clasificación con KNN y Random Forest:** Se implementa el algoritmo de *K-Nearest Neighbors* con un procesamiento por lotes para mejorar la eficiencia y se construye un *Random Forest* propio a partir de árboles

de decisión simples. Posteriormente, se comparan con las implementaciones de scikit-learn.

Este análisis permite no solo utilizar modelos de aprendizaje supervisado, sino también comprender las matemáticas detrás de los algoritmos, estructuras de datos y procesos de optimización que los sustentan. De esta manera, se refuerzan las habilidades prácticas y teóricas necesarias para la aplicación de este tipo de soluciones en problemas reales.

II. MARCO TEÓRICO

El aprendizaje automático o también conocido como Machine Learning se ha consolidado como una de las ramas más relevantes de la inteligencia artificial, al permitir la construcción de modelos que aprenden patrones a partir de datos y realizan predicciones sin necesidad de ser programados explícitamente. En este proyecto se consideran tanto algoritmos de clasificación como de regresión, siendo estos los siguientes:

A. K-Nearest Neighbors (KNN)

El algoritmo de los k vecinos más cercanos es un modelo de clasificación basado en instancias. Para clasificar una muestra desconocida, se calculan sus distancias respecto a las muestras de entrenamiento y se seleccionan los k vecinos más próximos. La clase final se asigna por votación mayoritaria. El método depende fuertemente de la métrica utilizada, siendo la distancia euclidiana la más común y la utilizada en este proyecto para este algoritmo:

$$d(x_i, x_j) = \sqrt{\sum_{i=0}^n (X_i - Y_i)^2}$$

B. Regresión Logística

La regresión logística es un modelo paramétrico usado en problemas de clasificación binaria. A diferencia de la regresión lineal, utiliza la función sigmoide para mapear cualquier valor real al rango $(0, 1)$, interpretado como probabilidad:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

donde $z = w^T x + b$ corresponde a la combinación lineal de las variables de entrada. El modelo se entrena mediante la minimización de la función de costo logarítmica (log-loss) usando gradiente descendente.

C. Árboles de Decisión

Los árboles de decisión son modelos que dividen el espacio de características en regiones homogéneas mediante reglas de decisión. En clasificación, se utiliza como criterio de partición la **impureza de Gini**:

$$Gini(D) = 1 - \sum_{i=1}^C p_i^2$$

donde p_i es la proporción de instancias de la clase i en el conjunto D . En regresión, en lugar de impureza, se busca minimizar la varianza de los valores en cada partición. Los árboles son fáciles de interpretar, pero tienden a sobreajustar si no se controla su profundidad.

D. Random Forest

Random Forest es un método que combina múltiples árboles de decisión para mejorar la capacidad predictiva y reducir el sobreajuste. En clasificación, el resultado se obtiene mediante votación mayoritaria, mientras que en regresión se utiliza el promedio de las predicciones.

E. Regresión Lineal

La regresión lineal es un modelo clásico utilizado para estimar relaciones entre variables dependientes e independientes. Su entrenamiento se realiza mediante el método de mínimos cuadrados ordinarios (OLS), cuyo objetivo es minimizar el error cuadrático medio (MSE):

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

F. Optimización de Hiperparámetros

Muchos de los modelos estudiados requieren seleccionar parámetros óptimos, como la cantidad de vecinos en KNN, la profundidad máxima en árboles de decisión o el número de estimadores en Random Forest. Para este propósito se utiliza GridSearchCV, que realiza una búsqueda en un espacio de parámetros evaluando el desempeño del modelo.

En conjunto, estos modelos permiten abordar problemas de clasificación y regresión desde perspectivas distintas, lo que enriquece la comprensión de sus fundamentos y el análisis comparativo de sus resultados.

III. DETALLES DEL DISEÑO DEL PROGRAMA Y DESARROLLO

En esta sección se describen los algoritmos implementados, mostrando tanto su estructura en forma de diagramas de flujo.

A. K-Nearest Neighbors (KNN)

El algoritmo KNN se basa en el cálculo de distancias y la votación entre los k vecinos más cercanos. Para nuestro caso se utilizó $k = 5$, ya que se observó que este valor ofrecía un equilibrio entre sesgo y varianza.

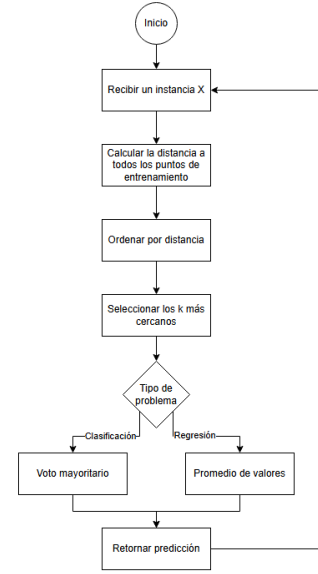


Fig. 1. Diagrama de flujo del algoritmo KNN.

B. Árboles de Decisión

Los árboles de decisión se construyen dividiendo recursivamente el conjunto de datos. Se utilizó una profundidad máxima de 10, un mínimo de 20 muestras para dividir y al menos 5 muestras por hoja.

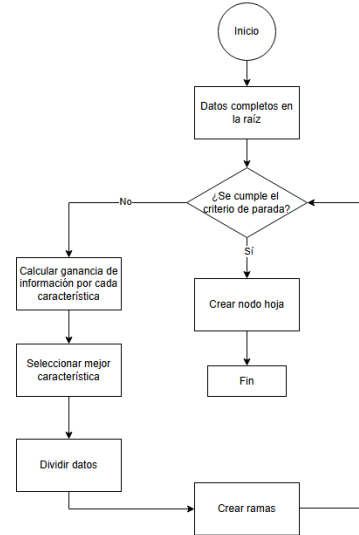


Fig. 2. Diagrama de flujo del algoritmo de árboles de decisión.

C. Random Forest

Para reducir el sobreajuste, se entrenaron múltiples árboles con el método de *bagging*. En clasificación se usaron 200 árboles y en regresión 100 árboles. Los hiperparámetros más relevantes fueron:

- **n_estimators**: número de árboles (200 para clasificación, 100 para regresión).
- **max_depth**: límite de profundidad para evitar sobreajuste.
- **min_samples_split**: número mínimo de muestras para dividir un nodo.

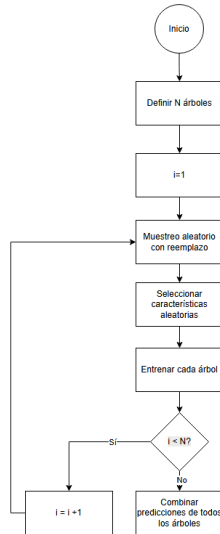


Fig. 3. Diagrama de flujo del algoritmo Random Forest.

D. Regresión Logística

Se entrenó mediante gradiente descendente, con una tasa de aprendizaje $\alpha = 0.01$ y un máximo de 1000 iteraciones. Se aplicó la función sigmoide para calcular las probabilidades.

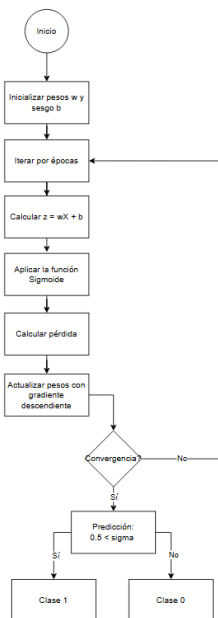


Fig. 4. Diagrama de flujo del algoritmo de regresión logística.

E. Regresión Lineal

El diagrama de flujo para la regresión lineal quedaría de la siguiente manera:

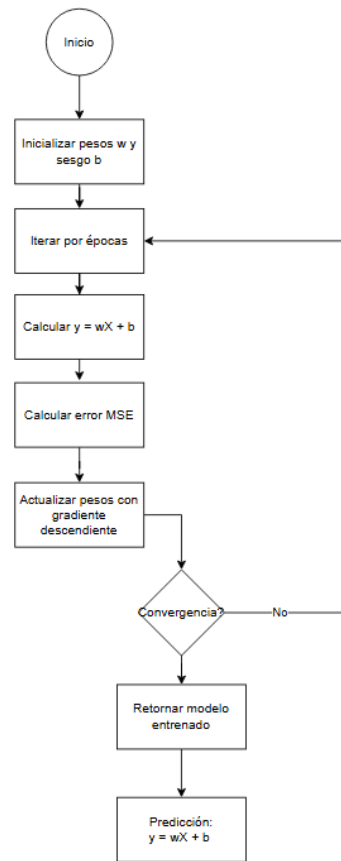


Fig. 5. Diagrama de flujo del algoritmo regresión lineal.

IV. RESULTADOS

A. Problema 1: Ingresos promedio por año

Se evaluaron tanto las implementaciones propias como las de sklearn, tanto con parámetros por defecto como tras un proceso de optimización con GridSearchCV. Las métricas reportadas incluyen Accuracy, Precision, Recall y F1-Score.

Modelo	Accuracy	Precision	Recall	F1-Score
Reg. Logística Propia	0.7804	0.5349	0.9021	0.6716
Árbol Dec. Propio	0.8034	0.5692	0.8655	0.6867
Reg. Logística Sklearn	0.8163	0.5910	0.8522	0.6979
Árbol Dec. Sklearn	0.8032	0.5689	0.8662	0.6867

TABLE I

COMPARACIÓN DE MODELOS CON PARÁMETROS POR DEFECTO.

1) *Comparación con parámetros por defecto*: El mejor desempeño en términos de Accuracy (0.8163) y F1-Score (0.6979) lo alcanzó la Regresión Logística de Sklearn.

Modelo	Accuracy	Precision	Recall	F1-Score
Reg. Log. Propia (Opt.)	0.8149	0.5883	0.8542	0.6967
Árbol Dec. Propio (Opt.)	0.8009	0.5896	0.6591	0.6224
Reg. Log. Sklearn (Opt.)	0.8165	0.5906	0.8569	0.6993
Árbol Dec. Sklearn (Opt.)	0.7989	0.5871	0.6485	0.6163

TABLE II
COMPARACIÓN DE MODELOS TRAS OPTIMIZACIÓN DE HIPERPARÁMETROS.

2) *Comparación con parámetros optimizados:* Con parámetros ajustados, el mejor resultado lo mantiene la Regresión Logística de Sklearn, alcanzando un Accuracy de 0.8165 y F1-Score de 0.6993.

3) *Matrices de confusión y curvas:* A continuación se presentan las gráficas más importantes del problema:

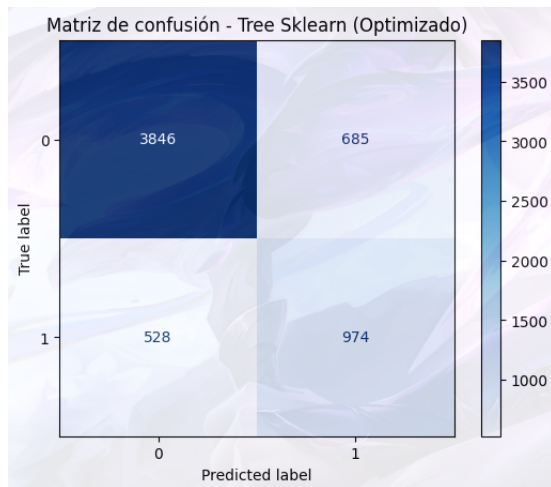


Fig. 6. Matriz de confusión para el Árbol de Decisión de Sklearn optimizado.

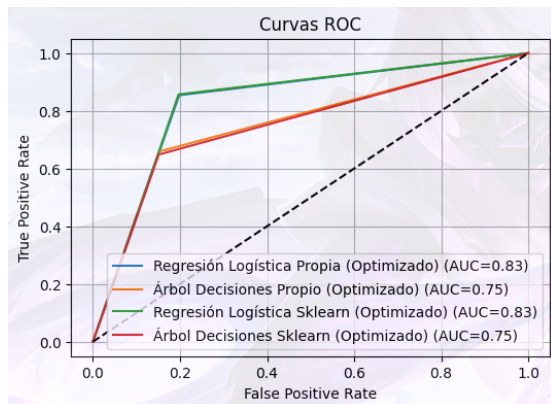


Fig. 7. Curvas ROC.

Los resultados muestran que la regresión logística mantiene un mejor equilibrio, con un área bajo la curva (AUC) de 0.83, superior al 0.75 del árbol de decisión. Esto sugiere que el modelo logístico es más adecuado para este conjunto de datos, tanto en su versión propia como en la de sklearn.

B. Problema 2: Calidad de aire

En este problema se compararon dos enfoques de regresión: la Regresión Lineal y el Random Forest, evaluando tanto implementaciones propias como las de la librería sklearn. Las métricas consideradas fueron MAE, MSE, RMSE y R^2 .

Modelo	MAE	MSE	RMSE	R^2
Regresión Lineal Propia	0.2939	0.2081	0.4562	0.8993
Regresión Lineal Sklearn	0.2939	0.2081	0.4562	0.8993

TABLE III
COMPARACIÓN DE REGRESIÓN LINEAL (IMPLEMENTACIÓN MANUAL VS. SKLEARN).

1) *Resultados Regresión Lineal:* Los resultados muestran que la implementación manual coincide exactamente con la de sklearn, validando la correcta construcción del modelo. Las gráficas de dispersión muestran que las predicciones siguen de cerca la línea de identidad, aunque con ligera dispersión en valores altos.

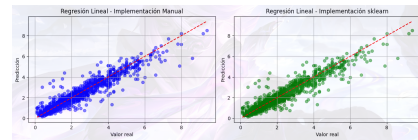


Fig. 8. Predicciones de la Regresión Lineal (implementación manual y sklearn).

Modelo	MAE	MSE	RMSE	R^2
Random Forest Propio (Opt.)	0.2531	0.1626	0.4032	0.9213
Random Forest Sklearn (Opt.)	0.2333	0.1458	0.3819	0.9294
Random Forest Sklearn (No Opt.)	0.2350	0.1479	0.3845	0.9285

TABLE IV
COMPARACIÓN DE RANDOM FOREST (IMPLEMENTACIÓN MANUAL VS. SKLEARN, OPTIMIZADO Y NO OPTIMIZADO).

2) *Resultados Random Forest:* El Random Forest superó claramente a la regresión lineal en todas las métricas, con un R^2 cercano a 0.93 en su versión optimizada de sklearn. La optimización de hiperparámetros redujo el MSE en un 1.37% y mejoró el R^2 en 0.11%.

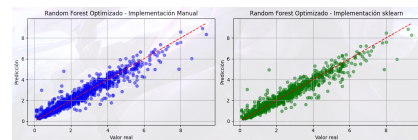


Fig. 9. Predicciones del Random Forest (implementación manual y sklearn).

La Regresión Lineal explica aproximadamente el 90% de la variabilidad de los datos, siendo útil para relaciones lineales. Sin embargo, el Random Forest demostró ser más potente, alcanzando un R^2 cercano al 93% y mostrando menor error en las métricas. La comparación entre implementaciones propias y sklearn demostró consistencia, aunque la librería ofrece ligeras mejoras gracias a sus optimizaciones internas.

C. Problema 3: KNN y Random Forest

En este problema se evaluaron dos algoritmos de clasificación: **K-Nearest Neighbors (KNN)** y **Random Forest (RF)**, comparando implementaciones propias y de la librería sklearn. Además de las métricas de clasificación, se incluyeron los tiempos de entrenamiento y predicción para analizar la eficiencia de cada versión.

1) *Métricas*: El resultado de las métricas son las siguientes:

Modelo	Accuracy	F1-Score	Precision	Recall
RF Sklearn	0.9995	0.9995	1.0000	0.9990
RF Propio	0.9989	0.9989	1.0000	0.9979
KNN Sklearn	0.9962	0.9964	0.9980	0.9949
KNN Propio	0.9978	0.9980	0.9990	0.9969

TABLE V

COMPARACIÓN DE MÉTRICAS Y TIEMPOS DE KNN Y RANDOM FOREST (IMPLEMENTACIÓN PROPIA VS. SKLEARN).

2) *Análisis*: Los resultados muestran que ambos algoritmos alcanzan un rendimiento extraordinariamente alto, con **Accuracy superior al 99.6%** en todas las variantes. Sin embargo, se observan claras diferencias en términos de eficiencia:

- **Random Forest de Sklearn** fue el mejor modelo, alcanzando un Accuracy de 0.9995 y F1-Score de 0.9995, con tiempos de entrenamiento y predicción mucho más bajos que su contraparte manual.
- **Random Forest propio** logró métricas muy similares, pero con un tiempo de entrenamiento extremadamente alto (más de 1900 segundos), lo que lo hace impráctico para aplicaciones reales.
- **KNN Sklearn** presentó el entrenamiento más rápido (0.05 s), ya que este algoritmo no requiere fase de ajuste intensivo, aunque la predicción resultó más lenta (1.99 s) al depender del cálculo de distancias.
- **KNN propio** obtuvo métricas ligeramente mejores que la versión de sklearn, pero con un tiempo de predicción de más de 790 segundos, confirmando que la optimización computacional es crucial en implementaciones reales.

V. ANÁLISIS

A. Problema 1

El conjunto de datos de ingresos presentaba un fuerte desbalance hacia la clase de personas con ingresos menores a 50k dólares. Para corregirlo se aplicó una estrategia combinada de oversampling y undersampling, con el fin de equilibrar la distribución de clases. Además, se eliminaron columnas irrelevantes usando la matriz de correlación, y los valores representados con el símbolo “?” fueron convertidos a NaN, eliminando posteriormente las filas con valores faltantes.

En cuanto a los modelos, no se observaron problemas significativos de overfitting, pues el desempeño fue estable entre entrenamiento y prueba. El árbol de decisión, sin embargo, mostró ligera tendencia a sobreajustarse a la clase mayoritaria, mientras que la regresión logística generalizó mejor.

B. Problema 2

El dataset contenía columnas con formato de fecha que fueron procesadas. Se aplicó interpolación para completar valores faltantes y se utilizó la matriz de correlación para descartar variables con correlación muy baja respecto al target.

La regresión lineal mostró un desempeño correcto pero limitado por la naturaleza lineal del modelo. El Random Forest, en cambio, logró capturar mejor las relaciones no lineales, alcanzando un mayor R^2 . No se detectaron problemas de sobreajuste gracias al uso de parámetros de regularización `max_depth`, `min_samples_split`, lo que permitió una buena generalización.

C. Problema 3

El dataset consistía en imágenes que fueron procesadas para reducir su dimensionalidad. Se aplicó un tratamiento binario para eliminar información irrelevante como sombras, y posteriormente se empleó PCA que fue la mejor forma. El mejor resultado se obtuvo con 200 componentes principales, balanceando reducción de dimensión y preservación de la información.

En este caso, la dimensionalidad reducida mejoró significativamente los tiempos de cómputo y la eficiencia de los algoritmos. Los modelos alcanzaron un rendimiento sobresaliente (Accuracy cercano al 100%), lo que indica que no hubo problemas de underfitting, aunque los tiempos de ejecución mostraron claras diferencias entre implementaciones propias y optimizadas.

VI. CONCLUSIONES

A lo largo de este trabajo se implementaron, optimizaron y compararon diferentes algoritmos de aprendizaje automático aplicados a problemas de clasificación y regresión. De los resultados obtenidos se desprenden las siguientes conclusiones principales:

- **Regresión Logística vs Árboles de Decisión**: La regresión logística mostró mejor capacidad de generalización y un equilibrio más estable entre *precision* y *recall*, superando a los árboles de decisión que tendieron a sobreajustarse a la clase mayoritaria.
- **Regresión Lineal vs Random Forest**: Si bien la regresión lineal logró explicar cerca del 90% de la variabilidad, el Random Forest optimizado alcanzó un R^2 superior al 92%, reduciendo significativamente el error y capturando relaciones no lineales presentes en los datos.
- **KNN vs Random Forest en imágenes**: Ambos modelos alcanzaron una precisión sobresaliente (superior al 99%), aunque Random Forest de scikit-learn fue el más eficiente en términos de tiempo y desempeño. El uso de PCA con 200 componentes resultó fundamental para reducir dimensionalidad y acelerar el proceso.
- **Implementaciones propias vs scikit-learn**: Las implementaciones manuales reprodujeron correctamente el comportamiento esperado de los algoritmos, validando su comprensión teórica. Sin embargo, los tiempos de

cómputo fueron considerablemente mayores en comparación con las versiones optimizadas de scikit-learn.

En síntesis, los modelos propuestos por sklearn tienen mejores resultados en términos de métricas, sin embargo la implementación propia brinda el conocimiento de lo que hay por detrás de estos y el cómo lograr optimizarlos y modificarlos a gusto más en caso de que se requiera en trabajos futuros.

REFERENCES

- [1] IBM, *¿Qué es el algoritmo KNN?*, IBM Think Blog. Disponible en: <https://www.ibm.com/mx-es/think/topics/knn>. [Accedido: 01-oct-2025]
- [2] IBM, *¿Qué es el bosque aleatorio?*, IBM Think Blog. Disponible en: <https://www.ibm.com/mx-es/think/topics/random-forest>. [Accedido: 01-oct-2025]
- [3] IBM, *Regresión lineal*, IBM Think Blog. Disponible en: <https://www.ibm.com/docs/es/db2/11.5.x?topic=building-linear-regression>. [Accedido: 01-oct-2025]
- [4] GeeksForGeeks, *Linear Regression in Machine learning*, GeeksForGeeks. Disponible en: <https://www.geeksforgeeks.org/machine-learning/ml-linear-regression/>. [Accedido: 01-oct-2025]
- [5] IBM, *¿Qué es la regresión logística?*, IBM Thinking Blog. Disponible en: <https://www.geeksforgeeks.org/machine-learning/ml-linear-regression/>. [Accedido: 01-oct-2025]

VII. REPOSITORIO DE GITHUB

https://github.com/cersluv/Proyecto1_AprendizajeSupervisado